

# GridPool V2.1 Modeling Findings

Variance, Incentives, Relay Latency, and Snapshot-Boundary Finality

GridPool research packet

2026-07-17

## Contents

|           |                                                                    |           |
|-----------|--------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Executive Summary</b>                                           | <b>2</b>  |
| <b>2</b>  | <b>Evidence Scope</b>                                              | <b>2</b>  |
| <b>3</b>  | <b>Boundaries Of The Current Evidence</b>                          | <b>3</b>  |
| <b>4</b>  | <b>Claims, Mechanisms, Evidence, And Caveats</b>                   | <b>3</b>  |
| <b>5</b>  | <b>Threat Model Summary</b>                                        | <b>4</b>  |
| <b>6</b>  | <b>V2.1 Snapshot/Reserve Mechanics</b>                             | <b>4</b>  |
| <b>7</b>  | <b>How V2.1 Reframes Earlier Attack Models</b>                     | <b>6</b>  |
| 7.1       | Latency Splits . . . . .                                           | 6         |
| 7.2       | Adversarial Splits And Majority Hashrate . . . . .                 | 6         |
| 7.3       | Work Set Conflicts . . . . .                                       | 6         |
| 7.4       | Concrete Split/Merge Example . . . . .                             | 6         |
| <b>8</b>  | <b>Finding 1: V2.1 Boundary-Inclusion Loss Is Bounded</b>          | <b>7</b>  |
| <b>9</b>  | <b>Finding 2: 300-Slot Payout Lists Reduce Actual BTC Variance</b> | <b>8</b>  |
| <b>10</b> | <b>Finding 3: Pool Hopping Cannot Create Fake Work</b>             | <b>9</b>  |
| <b>11</b> | <b>Finding 4: Block Withholding Looks Expensive</b>                | <b>10</b> |
| <b>12</b> | <b>Finding 5: Compact Relay Still Matters Under V2.1</b>           | <b>11</b> |
| <b>13</b> | <b>Finding 6: Merge Valid Work, Finalize Snapshot Boundaries</b>   | <b>12</b> |
| <b>14</b> | <b>Mathematical Appendix: Top Shares And Snapshot Survival</b>     | <b>13</b> |
| <b>15</b> | <b>Regression Test Summary</b>                                     | <b>14</b> |
| <b>16</b> | <b>Remaining Engineering Work</b>                                  | <b>15</b> |
| <b>17</b> | <b>Economic Intuition</b>                                          | <b>16</b> |
| <b>18</b> | <b>Reproducibility Appendix</b>                                    | <b>16</b> |
| 18.1      | Payout Variance . . . . .                                          | 16        |
| 18.2      | Pool Hopping . . . . .                                             | 16        |
| 18.3      | Block Withholding . . . . .                                        | 17        |
| 18.4      | Relay Latency . . . . .                                            | 17        |
| 18.5      | Consensus Scoring Audit . . . . .                                  | 17        |

|                                              |           |
|----------------------------------------------|-----------|
| 18.6 V2.1 Boundary-Inclusion Model . . . . . | 18        |
| <b>19 Public Network Summary</b>             | <b>18</b> |
| <b>20 Resource Roadmap</b>                   | <b>19</b> |
| <b>21 Source Pointers</b>                    | <b>20</b> |

## 1 Executive Summary

GridPool replaces trusted pool accounting with independently verifiable proof-of-work claims against explicit payout snapshots. The current modeling evidence supports three core results:

- broad payout lists materially reduce actual BTC payout variance;
- pool hopping cannot create fake work or steal another miner’s earned proof claims;
- straightforward profit-seeking block withholding is expensive because the finder gives up slot 0, transaction fees, and acceleration of its own shared claims.

The most important protocol update is V2.1 snapshot/reserve consensus. Earlier modeling framed split recovery as a branch-selection problem: if two nodes disagreed, which proof list was “heavier”? That framing exposed real concerns around delayed stale branches and adversarial same-boundary scoring.

V2.1 changes the launch rule:

Merge valid compatible work forward. Do not retroactively rewrite a locally observed Bitcoin-block payout snapshot with late previous-parent work.

Under this rule, most honest Work Set divergence is synchronization, not consensus branch selection. Nodes validate compatible current-parent proofs, merge them into the unpaid Work Set, sort by achieved difficulty, and retain the bounded reserve. Late previous-parent proofs that arrive after a node has observed the next Bitcoin block are rejected or quarantined from canonical state rather than used to rewrite the already-active payout snapshot.

This does not make GridPool immune to network partitioning, denial-of-service, censorship, or grieving by miners with external motives. It does, however, remove the clean stale-branch entry path assumed by older “heaviest stale branch wins” models while preserving honest split recovery through merge-forward Work Sets.

## 2 Evidence Scope

| Topic               | Current evidence                                                                                                               | Status                                                                                             |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| V2.1 split handling | Protocol analysis and implementation direction: merge compatible current-parent Work Sets; finalize local snapshot boundaries. | Coherent launch rule; still needs regression tests and multi-node soak.                            |
| Payout variance     | Reproducible variance sweep using actual BTC paid, not just slot inclusion.                                                    | Strong model-backed result under stated assumptions.                                               |
| Pool hopping        | Economic sweeps plus explicit proof-claim mechanics.                                                                           | Strong mechanical result; small/caveated economic effect under idealized zero-fee outside options. |
| Block withholding   | Fee-sensitivity sweep across honest, withhold-all, and selective withholding.                                                  | Strong model-backed evidence against straightforward profit-seeking withholding.                   |
| Relay latency       | Network latency model and generated charts.                                                                                    | Useful model evidence; live public-node telemetry remains preliminary.                             |

| Topic                   | Current evidence                                                    | Status                                                     |
|-------------------------|---------------------------------------------------------------------|------------------------------------------------------------|
| Older consensus scoring | Honest/adversarial scoring audit and delayed-snapshot attack model. | Historical rationale for V2.1; not the active launch rule. |

### 3 Boundaries Of The Current Evidence

| Current evidence supports                                                                                   | Current evidence does not establish                                                        |
|-------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Verifiable payout snapshots can replace trusted pool accounting for modeled flows.                          | Immunity to every majority-miner, partition, censorship, or denial-of-service scenario.    |
| 300 shared payout slots materially reduce actual BTC payout variance once team block cadence is meaningful. | Smooth near-term payouts for tiny miners on tiny teams.                                    |
| Pool hopping cannot create unpaid shares, forge work, or steal earned proof claims.                         | That every outside-pool fee, liquidity, and variance condition makes hopping irrational.   |
| Straightforward profit-seeking block withholding underperforms honest mining in the current model.          | That grieving is impossible when the attacker has external utility.                        |
| Compact relay reduces modeled snapshot-boundary disagreement risk relative to JSON relay.                   | A production-measured global split-risk reduction from UDP/compact relay.                  |
| V2.1 prevents late previous-parent proofs from rewriting already-active snapshots.                          | A final answer to every future scoring, admission-control, and network-partition question. |

### 4 Claims, Mechanisms, Evidence, And Caveats

| Claim                                                         | Mechanism                                                                                                           | Evidence                                                                              | Caveat                                                                                    |
|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| 300-slot payout lists reduce actual BTC payout variance.      | Team blocks pay broad shared snapshots; expected BTC stays fair while payouts are less clumped.                     | Payout variance sweep: <del>reports/generated/sweeps/payout_variance_file_long/</del> | Tiny miners on tiny teams until team block cadence is meaningful.                         |
| Pool hopping cannot create fake work or steal earned value.   | Shares are proof-bound, validated, and paid once by proof ID.                                                       | Protocol mechanics plus pool-hopping sweeps.                                          | Zero-fee outside options can create a small free-option effect for already-earned claims. |
| Block withholding is unattractive for profit-seeking miners.  | Withholding gives up slot 0, transaction fees, and acceleration of the attacker's own shared claims.                | Block-withholding fee-sensitivity sweep.                                              | External-motive grieving is outside the profit-maximizing model.                          |
| Compact relay reduces modeled snapshot-boundary disagreement. | Lower share propagation delay increases the chance that proofs arrive before the Bitcoin-block boundary.            | V2.1 boundary-inclusion model plus latency peer-degree sweep.                         | This is model evidence; live public-node telemetry is preliminary.                        |
| V2.1 prevents late stale-parent snapshot rewrites.            | Late previous-parent proofs after local Bitcoin-block observation are rejected or quarantined from canonical state. | V2.1 protocol rule and delayed-snapshot model rationale.                              | Network partitions, censorship, and denial-of-service remain operational risks.           |

| Claim                                                                                     | Mechanism                                                                                                        | Evidence                                      | Caveat                                                             |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|--------------------------------------------------------------------|
| Latency becomes bounded inclusion risk rather than continuous sharechain orphan pressure. | Compatible current-parent proofs merge forward; stale previous-parent proofs do not rewrite finalized snapshots. | V2.1 split/merge analysis plus latency model. | Fast relay still matters for inclusion before snapshot boundaries. |

## 5 Threat Model Summary

| Threat                    | V2.1 Mechanism                                                                                    | Current Evidence                                           | Still Open                                                                              |
|---------------------------|---------------------------------------------------------------------------------------------------|------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Pool hopping              | Earned proof claims remain proof-bound and paid once.                                             | Pool-hopping simulations and proof-lineage mechanics.      | Outside-pool fees, payout timing, liquidity, and variance can change rational behavior. |
| Block withholding         | Finder loses slot 0, transaction fees, and faster payment of its own shared claims.               | Fee-sensitivity withholding sweep.                         | Griefing with external utility is not ruled out.                                        |
| Stale branch rewrite      | Late previous-parent proofs cannot rewrite a locally finalized snapshot.                          | Delayed-snapshot attack model motivates the boundary rule. | Runtime needs regression tests and multi-node soak evidence.                            |
| Majority miner censorship | Majority miner cannot forge others' proofs or erase paid lineage through stale rewrite.           | Protocol mechanics and adversarial model framing.          | A majority can censor locally observed shares, split off, or degrade UX.                |
| Network partition         | Compatible work can merge after reconnection if proofs validate against retained contexts.        | V2.1 merge-forward rule.                                   | Long partitions and eclipse-style attacks require operational/network hardening.        |
| Latency disadvantage      | Late current-parent work can merge forward; late previous-parent work simply misses the boundary. | Latency model and V2.1 boundary semantics.                 | Real public-node latency distribution needs cleaner measurement.                        |
| State-bundle spam         | Candidate bundles must validate proof context and cannot smuggle stale work into canonical state. | V2.1 invariant design.                                     | Needs explicit adversarial bundle tests.                                                |
| External-motive griefing  | Protocol removes direct profit incentives for simple withholding and stale rewrites.              | Economic models.                                           | Attackers willing to burn money remain a broader network/security problem.              |

## 6 V2.1 Snapshot/Reserve Mechanics

V2.1 uses four separate state concepts:

| Term             | Meaning                                                                                                                                                |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unpaid Work Set  | The bounded reserve of valid, unpaid share proofs. Default reserve depth is 3x, so the current 299 shared-slot configuration retains up to 897 proofs. |
| Active Snapshot  | The payout template locked at a Bitcoin block boundary. A GridPool-found block pays this snapshot.                                                     |
| Snapshot Context | The payout template a proof was mined against. Nodes retain contexts so unpaid proofs can still be verified after later snapshots.                     |
| Paid Lineage     | The proof IDs paid by a real GridPool block. Paid proof IDs are removed once paid; unpaid reserve proofs remain eligible.                              |

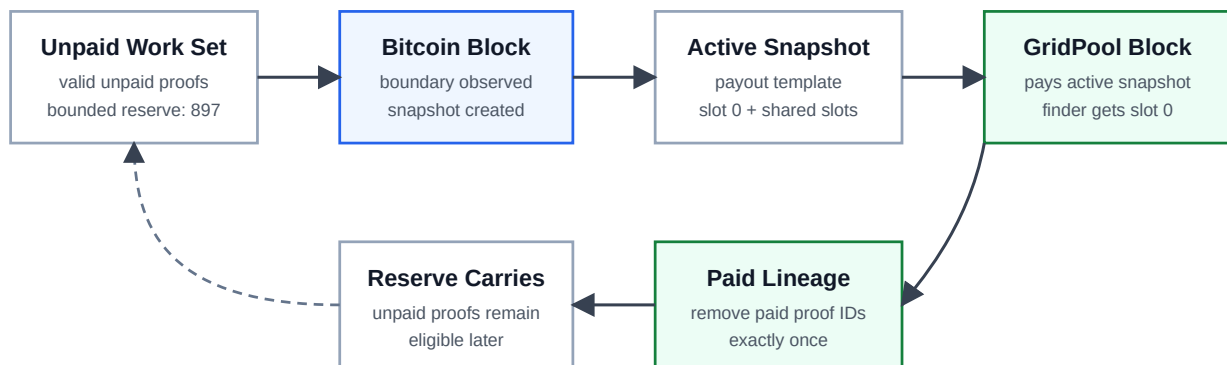
The operational rules are:

1. Every new Bitcoin block snapshots the current unpaid Work Set into the active payout template.
2. Snapshot creation does not clear the unpaid Work Set.
3. A real GridPool block pays the active snapshot.
4. After payment, remove only proof IDs that were actually paid.
5. If two peers have compatible valid Work Set proofs, merge them and keep the best bounded reserve.
6. If a previous-parent proof arrives after the local node has observed the next Bitcoin block, do not use it to rewrite the already-active snapshot.
7. A peer's heavier stale branch is not enough to retroactively replace a snapshot the local node already finalized.

The important distinction is merge versus replace. Honest nodes do not need to pick one whole Work Set and discard the other when both contain valid compatible current-parent work. They merge valid proofs.

## GridPool V2.1 Snapshot/Reserve State Transition

Snapshots lock payout templates; unpaid reserve proofs persist until they are actually paid.



**Boundary rule:** late previous-parent proofs after local Bitcoin-block observation do not rewrite the active snapshot.  
Compatible current-parent proofs can still merge forward into the unpaid Work Set after validation.

Figure 1: V2.1 snapshot/reserve state transition

## 7 How V2.1 Reframes Earlier Attack Models

### 7.1 Latency Splits

Earlier modeling treated last-millisecond shares as potential branch-selection events. If two nodes created different payout lists, a node might need to choose which list was canonical. That framing was too close to a P2Pool-style continuous race.

V2.1 narrows the latency cost:

- valid current-parent work can merge forward into the unpaid reserve;
- a missed last-second previous-parent proof may fail to enter that specific snapshot;
- the missed proof does not create an ongoing winner-take-all sharechain race;
- nodes do not reorganize an already-observed snapshot just because a peer later reports one more previous-parent proof.

GridPool still cares about propagation latency. Faster relay improves the chance that high-difficulty shares are seen before a Bitcoin-block snapshot boundary. The risk is bounded around snapshot inclusion rather than continuous sharechain orphaning.

### 7.2 Adversarial Splits And Majority Hashrate

Older models asked whether a miner with majority GridPool hashrate could privately mine a favorable stale branch and later convince others to follow it because it was heavier. That model assumed retroactive stale-branch replacement was allowed.

V2.1 removes the direct entry point for that attack:

- stale previous-parent work cannot rewrite a snapshot after the local node observed the next Bitcoin block;
- current-parent work can be merged forward instead of forcing a whole-branch choice;
- an attacker cannot forge another miner's proof or erase paid lineage by presenting a heavier stale branch;
- continuing a private stale branch gives up current-tip mining opportunities, including slot 0 and transaction fees.

A majority miner can still grief, censor shares it sees locally, isolate peers, partition the network, or split off into a separate team. Those are real risks. They are not the same as retroactively stealing or rewriting proof claims with a stale branch.

### 7.3 Work Set Conflicts

If two nodes share the same active snapshot but have different unpaid proofs, there is usually no consensus conflict. The expected behavior is:

1. validate each proof against its claimed parent and snapshot context;
2. discard duplicates and invalid proofs;
3. merge valid compatible proofs;
4. sort by achieved difficulty;
5. retain the top bounded reserve.

Most ordinary Work Set disagreement is mergeable data synchronization, not Byzantine branch selection.

### 7.4 Concrete Split/Merge Example

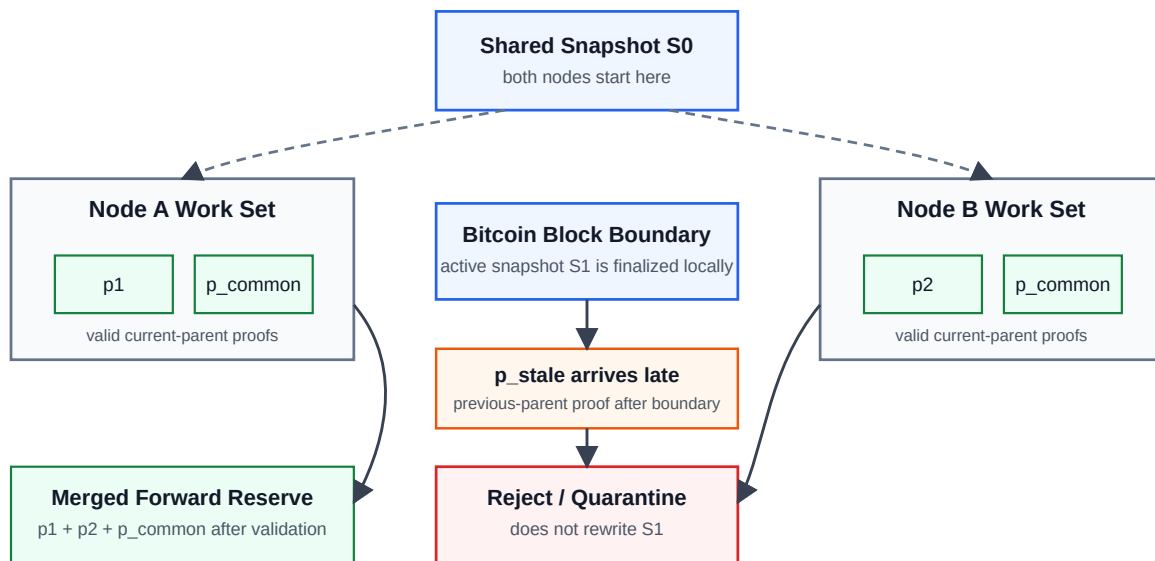
Suppose Node A and Node B start from the same active snapshot **S0**.

1. Before the next boundary, Node A sees valid proof **p1**; Node B sees valid proof **p2**; both know **p\_common**.
2. A Bitcoin block arrives, so each node creates an active snapshot **S1** from its locally known unpaid Work Set.
3. After that boundary, a previous-parent proof **p\_stale** arrives from a peer.
4. Under V2.1, **p\_stale** is rejected or quarantined from canonical state because it arrived too late to affect **S1**.
5. Valid compatible current-parent proofs **p1**, **p2**, and **p\_common** merge forward into the unpaid reserve after validation.

The result is not “Node A’s branch wins” or “Node B’s branch wins.” The result is that compatible valid work is merged, while the late previous-parent proof cannot rewrite the already-active snapshot.

## V2.1 Split/Merge Toy Example

Compatible current-parent proofs merge forward; late previous-parent work cannot rewrite a finalized snapshot.



Result: no whole-branch vote is needed for compatible work; only the late previous-parent proof is excluded from canonical state.

Figure 2: V2.1 split/merge toy example

## 8 Finding 1: V2.1 Boundary-Inclusion Loss Is Bounded

The V2.1 boundary-inclusion model directly tests the revised consensus framing. Nodes receive shares with relay delays, a Bitcoin block creates a snapshot boundary, late previous-parent proofs are not allowed to rewrite the active snapshot, and compatible current-parent work remains mergeable into later Work Sets.

The long run used 24 nodes, peer degree 8, 299 shared slots, 5000 Bitcoin-block boundaries per replication, and 12 replications per relay profile. That is 60,000 modeled boundaries per profile.

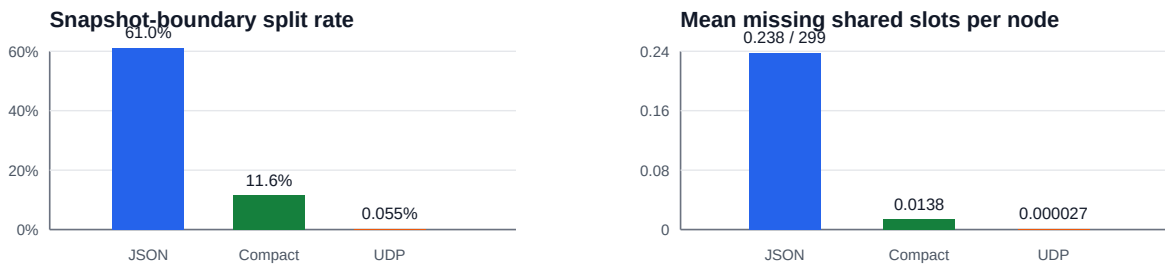
| Relay profile             | Split rate | Avg missing slots /<br>node | Slot-pair loss | Payload / day |
|---------------------------|------------|-----------------------------|----------------|---------------|
| JSON/HTTP                 | 61.0%      | 0.238 / 299                 | 0.0794%        | 10.9 MB/day   |
| Compact                   | 11.6%      | 0.0138 / 299                | 0.00461%       | 4.45 MB/day   |
| WebSocket                 |            |                             |                |               |
| UDP-fast with<br>fallback | 0.055%     | 0.000027 / 299              | 0.000009%      | 4.44 MB/day   |

The headline is intentionally nuanced. JSON/HTTP has a high boundary split rate, but the actual modeled loss is small: about 0.24 missing shared slots per node per boundary on average. Compact relay reduces split rate by about 5.25x and slot-pair loss by about 17.2x versus JSON/HTTP. UDP-fast relay reduces split rate by about 1100x and slot-pair loss by about 8770x versus JSON/HTTP.

This supports the V2.1 claim that latency is primarily a bounded snapshot-boundary inclusion risk, not a P2Pool-style continuous orphan race. Faster relay still matters, but because it reduces the already-bounded number of previous-parent proofs that miss a snapshot boundary.

## V2.1 Boundary Inclusion Loss By Relay Profile

24 nodes, degree 8, 60,000 Bitcoin-block boundaries per profile. Split rate can be high while actual slot loss remains bounded.



### Interpretation

JSON/HTTP has frequent boundary disagreement, but the modeled average loss is about 0.24 of 299 shared slots per node.

Compact relay cuts slot-pair loss by about 17.2x versus JSON; UDP-fast cuts it by about 8,770x.

Bandwidth is not the limiting factor in this scenario: about 10.9 MB/day for JSON and 4.4 MB/day for compact/UDP.

Figure 3: V2.1 boundary inclusion loss by relay profile

## 9 Finding 2: 300-Slot Payout Lists Reduce Actual BTC Variance

The variance model treats actual BTC paid as the miner utility target, not just slot inclusion. This distinction matters because if team hashrate doubles, a fixed-size miner receives a smaller fraction of slots per team block, but team blocks arrive more often. Expected BTC stays roughly the same; variance changes because payouts are less clumped.

Reviewed example, 1 PH miner at team multiplier 10000:

| Total payout slots | Variance reduction vs solo | Zero-payout probability |
|--------------------|----------------------------|-------------------------|
| 300                | about 271.7x               | about 11.9%             |
| 100                | about 96.7x                | about 48.8%             |
| 10                 | about 10.0x                | about 93.1%             |

The model supports the conclusion that a 300-slot payout list substantially reduces payout variance compared with smaller lists once the team finds blocks often enough for the miner to receive recurring shared payouts. Tiny miners on tiny teams remain lottery-like because expected paid events are still rare.



## Variance Reduction Versus Solo By Slot Count

Mean across miner sizes after applying the same filter as the CV chart.

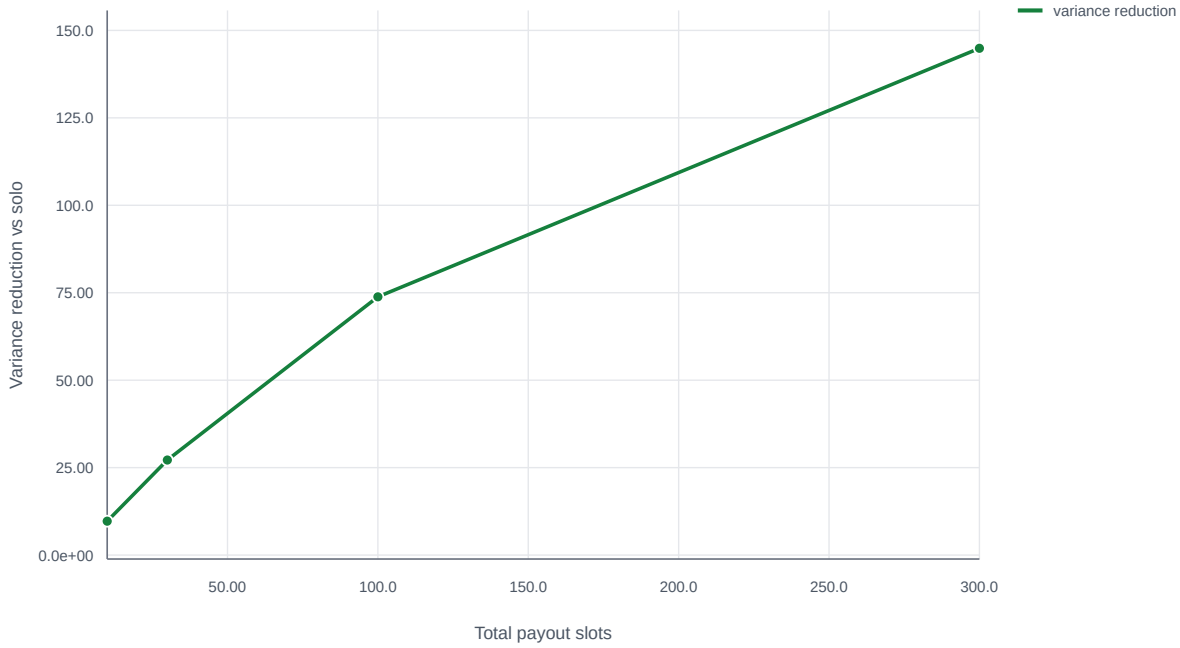


Figure 4: Variance reduction by payout list size

## 10 Finding 3: Pool Hopping Cannot Create Fake Work

The strongest result is mechanical: a miner can only retain claims to proof-of-work it already contributed. Hopping cannot mint unpaid shares or steal slots from other miners.

The economic result is more nuanced. With a zero-fee deterministic FPPS-like outside option, the model can show a small free-option effect because already-earned GridPool proofs remain payable while the miner earns elsewhere. In the reviewed sweep, zero-fee deterministic FPPS hopper EV ratios were roughly 1.0006 to 1.0020. At 0.5% outside fee, deterministic FPPS hopping fell below fair gross network EV; at 2%, it clearly underperformed.

The modeled free-option effect is not a way to forge work or steal shares. It is a consequence of already-earned proof claims remaining real claims. Realistic outside fees, variance, payout thresholds, stale policy, counterparty risk, and operational costs can erase or reverse the effect. Solo outside-option runs are especially noisy because variance changes sharply.

## Pool Hopping Paired Delta: deterministic\_fpps

Delta versus always-on GridPool on paired Monte Carlo seeds.

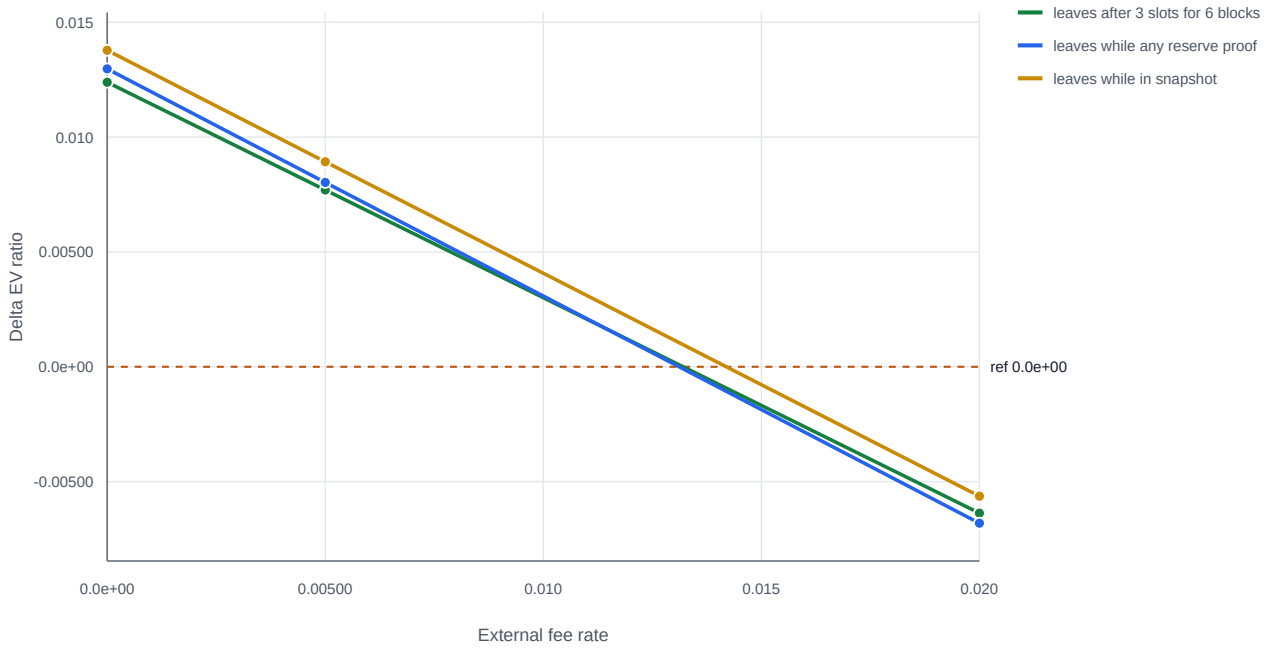


Figure 5: Pool hopping deterministic FPPS paired delta

## 11 Finding 4: Block Withholding Looks Expensive

The current model compares honest mining, withholding every valid GridPool block, and withholding only when the attacker is underrepresented in the active snapshot.

Attacker EV ratios for withhold-all:

| Transaction fees per block | Attacker EV ratio |
|----------------------------|-------------------|
| 0.0 BTC                    | about 0.8345      |
| 0.05 BTC                   | about 0.8214      |
| 0.25 BTC                   | about 0.7727      |
| 1.0 BTC                    | about 0.6322      |

In the current model, withholding valid GridPool blocks is costly because the finder gives up slot 0, transaction fees, and acceleration of its own existing shared payout claims. This does not prove grieving is impossible. It shows that straightforward profit-seeking withholding is unattractive under the modeled assumptions.

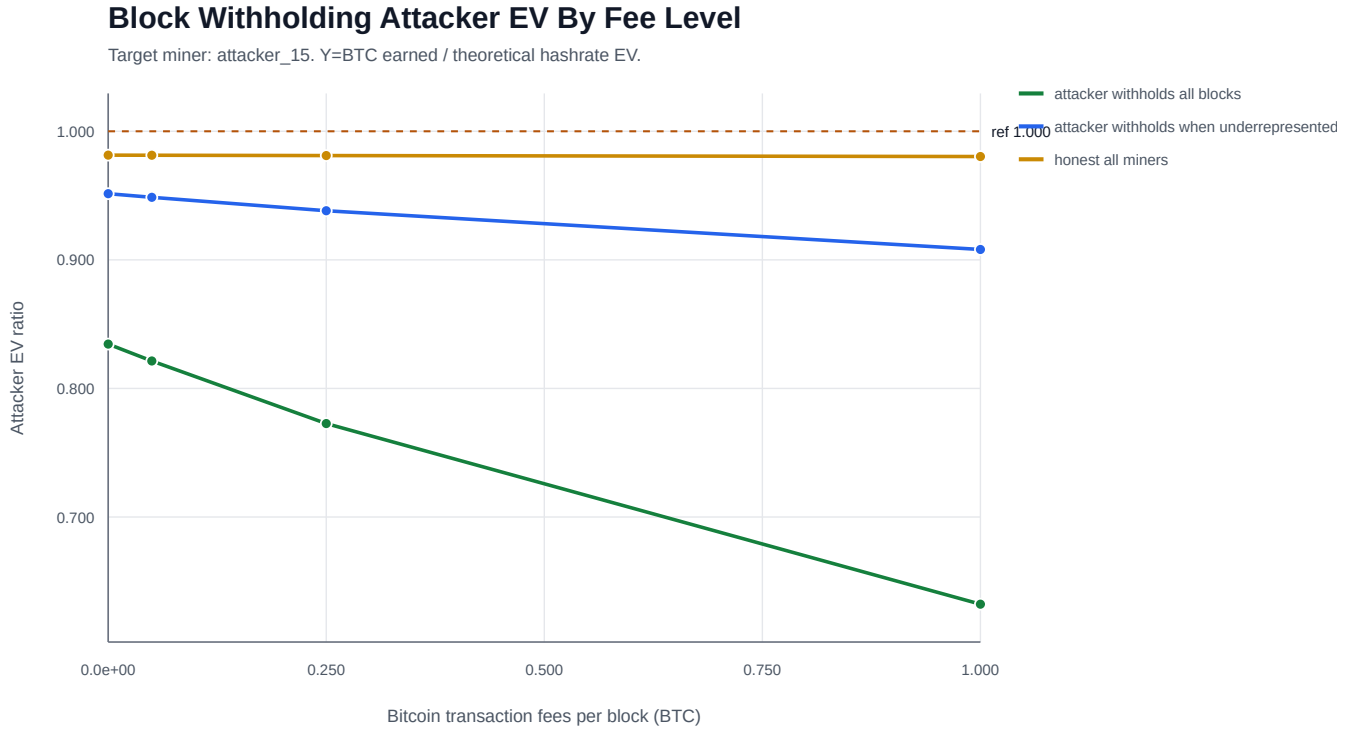


Figure 6: Block withholding attacker EV by fee level

## 12 Finding 5: Compact Relay Still Matters Under V2.1

V2.1 reduces the harm from latency splits, but it does not make latency irrelevant. Fast share propagation still improves the chance that high-difficulty shares are seen by peers before the next Bitcoin-block snapshot boundary.

The latency model compares three relay profiles:

| Profile                      | Share mean | Block mean | Payload |
|------------------------------|------------|------------|---------|
| JSON/HTTP relay              | 650 ms     | 850 ms     | 2200 B  |
| Compact WebSocket relay      | 190 ms     | 850 ms     | 900 B   |
| UDP-fast relay with fallback | 35 ms      | 850 ms     | 900 B   |

Selected degree-8 split rates:

| Nodes | JSON/HTTP | Compact WebSocket | UDP-fast |
|-------|-----------|-------------------|----------|
| 16    | 0.0050    | 0.00083           | 0.0      |
| 24    | 0.00542   | 0.00208           | 0.0      |
| 48    | 0.01333   | 0.00542           | 0.00083  |

The older relay model and the newer V2.1 boundary-inclusion model point in the same direction: compact relay materially reduces snapshot-boundary disagreements compared with full JSON relay, and UDP-fast relay reduces them further. The V2.1 model also shows why the result should not be overstated. Slower relay increases boundary inclusion loss, but it does not create a permanent branch-selection fight because compatible current-parent work remains mergeable.

## Snapshot Split Scaling At Degree 8



Figure 7: Snapshot split scaling at degree 8

## 13 Finding 6: Merge Valid Work, Finalize Snapshot Boundaries

The old consensus-scoring work remains useful, but its role has changed. It explains why V2.1 avoids retroactive “heaviest stale branch wins” replacement instead of relying on a perfect global branch-scoring rule.

Proof difficulty above a floor has a heavy Pareto tail:

$$P(D \geq x \mid D \geq \text{floor}) = \text{floor} / x$$

That tail means one monster proof can dominate raw summed difficulty even when it came from the smaller side of a split.

Honest-mode mean tie-adjusted accuracy from the wide run:

| Rule                    | Mean tie-adjusted accuracy |
|-------------------------|----------------------------|
| bottom_1_times_count    | about 87.64%               |
| p10_times_count         | about 87.29%               |
| log_sum                 | about 85.80%               |
| sum_workset_difficulty  | about 75.25%               |
| snapshot_sum_difficulty | about 73.91%               |

The statistical result remains useful: the lowest retained proof in a full reserve is a strong estimator of total observed work over comparable windows. But after V2.1, this is no longer the primary same-boundary conflict resolver for normal operation.

Updated interpretation:

- reserve-floor and order-statistic metrics remain useful for diagnostics, research, and suspicious-state analysis;
- a peer’s heavier stale branch should not retroactively replace a locally finalized snapshot;

- valid compatible current-parent proofs should merge whenever possible;
- non-mergeable stale work should be rejected or quarantined, not treated as a competing canonical branch.

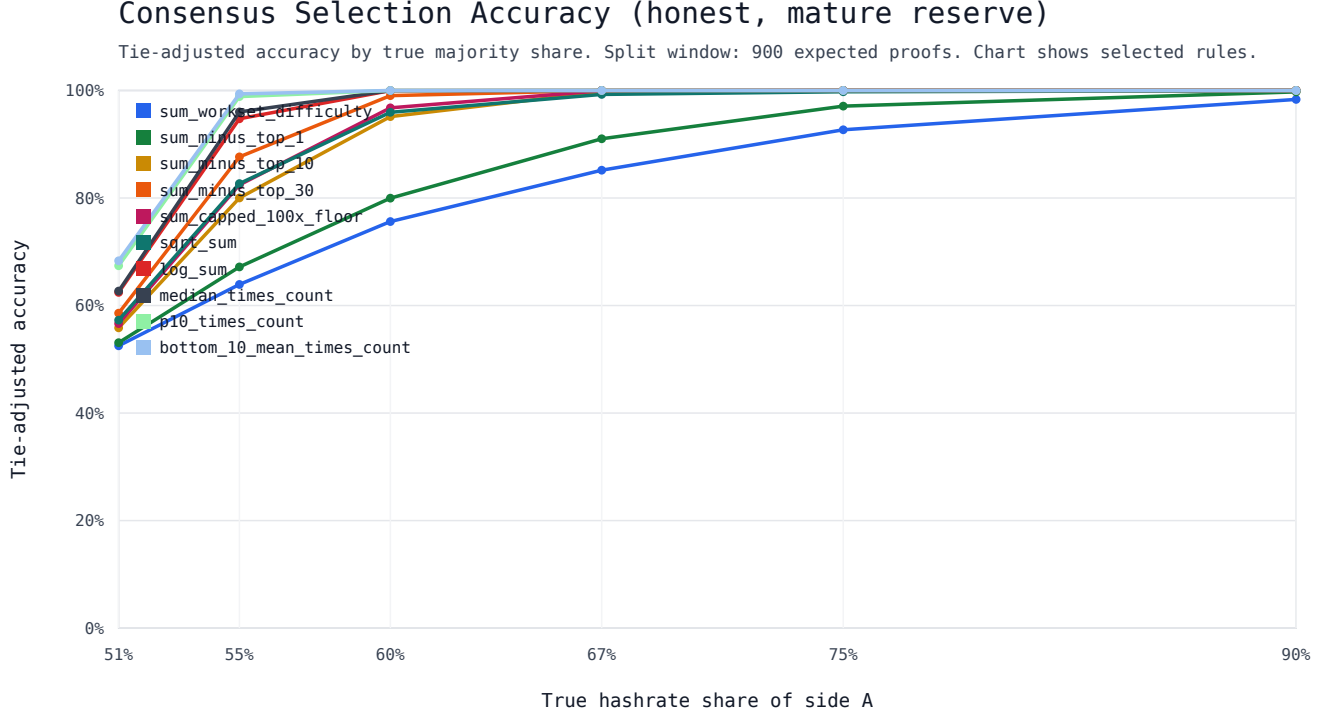


Figure 8: Honest mature-reserve scoring accuracy

## 14 Mathematical Appendix: Top Shares And Snapshot Survival

Share difficulty has a useful order-statistic structure. If  $D$  is the achieved difficulty of a share and  $d_{\min}$  is the admission floor, then

$$P(D \geq x \mid D \geq d_{\min}) = \frac{d_{\min}}{x}.$$

Equivalently, the normalized value

$$V = \frac{d_{\min}}{D}$$

is uniformly distributed on  $(0, 1)$ . The best shares are therefore the smallest order statistics  $V_{(1)} < \dots < V_{(m)}$ , or equivalently the largest achieved difficulties.

For a retained reserve of size  $m$ , the  $m$ -th best share carries the scale information needed to estimate the total number of submitted shares  $S$  over a comparable observation window:

$$\hat{S} \approx \frac{m}{V_{(m)}} = m \cdot \frac{D_{(m)}}{d_{\min}}.$$

The relative standard error is approximately  $1/\sqrt{m}$ . This is about 3.34% for  $m = 897$  and about 5.77% for  $m = 300$ . That is why the bottom of a full reserve is a meaningful estimator of observed work: one lucky monster proof has little effect on an  $m$ -order statistic when  $m$  is large.

The same math gives a useful intuition for whether a lucky share will survive into a payout snapshot. Suppose a share has achieved difficulty  $D_s$ , and the pool round ends when the team finds a Bitcoin block at network difficulty  $D_{\text{network}}$ . The number of better shares before the block is geometrically distributed, and the probability that this share remains in the top 300 shared slots is:

$$P(\text{survives top 300}) = 1 - \left( \frac{D_{\text{network}}}{D_s + D_{\text{network}}} \right)^{300}.$$

Using an illustrative early-July 2026 network difficulty of about 133.87 trillion:

| Share difficulty                        | Expected better shares<br>$D_{\text{network}}/D_s$ | Probability of surviving top 300 |
|-----------------------------------------|----------------------------------------------------|----------------------------------|
| 900B                                    | about 148.7                                        | about 86.6%                      |
| 576G                                    | about 232.4                                        | about 72.4%                      |
| $D_{\text{network}}/300 \approx 446.2B$ | 300.0                                              | about 63.2%                      |

The last row is initially surprising: even when the expected number of better shares is 300, the survival probability is above 60%. The reason is round-length variance. The block may arrive before the expected number of better shares accumulates.

This appendix supports two paper claims. First, large reserves make top-share order statistics useful for estimating observed work without trusting an operator share ledger. Second, lucky high-difficulty shares can have a substantial chance of remaining payable, but one lucky share should not dominate branch selection or retroactively rewrite a finalized snapshot.

## 15 Regression Test Summary

The runtime test suite was inspected in `boot-protocol` and run successfully on July 10, 2026:

```
dotnet test boot.tests/boot.tests.csproj
```

Result: Passed: 92, Failed: 0, Skipped: 0.

The tests are not a substitute for public multi-node soak data, but they do cover the core V2.1 invariants at the state-service and controller-harness level.

| Invariant                            | Why It Matters                                                                                                | Test Status                                | Source / Next Step                                                                                                                                                                                                                                                       |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Paid-once lineage                    | A proof should pay at most once; otherwise GridPool would inflate claims across lucky streaks.                | Covered by unit/integration harness tests. | <code>GridPoolPaymentRemovesOnlyPaidSnap</code><br><code>ConsecutiveGridPoolBlocksWalkDeepe</code><br><code>SupportFeePaymentRemovesOnlyActual</code><br>in<br><code>boot.tests/ShareAttributionTests.c</code><br>Next: confirm in multi-node soak after real peer sync. |
| Merge compatible current-parent work | Honest nodes can recover from divergent Work Sets without choosing one whole branch and discarding the other. | Covered for candidate-state import.        | <code>CandidateImportMergesCurrentParent</code><br>in<br><code>boot.tests/ShareAttributionTests.c</code><br>Next: add a full two-process peer sync regression when the peer harness is mature.                                                                           |

| Invariant                                              | Why It Matters                                                                                                                                                        | Test Status                                                                                                                             | Source / Next Step                                                                                                                                                                                                                                                                                                                                                             |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reject/quarantine late previous-parent rewrite         | Prevents stale-branch replacement after a local Bitcoin-block snapshot boundary.                                                                                      | Covered for candidate-state import.                                                                                                     | <code>CandidateImportIgnoresLatePrevious</code> in <code>boot.tests/ShareAttributionTests.c</code><br>Next: add an explicit peer-session bundle case and log/assert quarantine diagnostics.                                                                                                                                                                                    |
| Preserve unpaid reserve across Bitcoin-block snapshots | Ordinary Bitcoin blocks should update the active payout snapshot without destroying unpaid work.                                                                      | Covered.                                                                                                                                | <code>BitcoinBlockSnapshotUpdatesActiveW</code> in <code>boot.tests/ShareAttributionTests.c</code>                                                                                                                                                                                                                                                                             |
| Remove paid proof IDs exactly once                     | A GridPool block should remove paid snapshot proofs, keep unpaid reserve proofs, and ignore duplicate block application.                                              | Covered.                                                                                                                                | Payment removal tests above, plus duplicate block application assertion around "Block already applied" in <code>boot.tests/ShareAttributionTests.c</code><br>Next: add an explicit named idempotence test if this becomes part of the public spec test vector set.                                                                                                             |
| Support-fee on/off payout validation                   | The optional canonical support slot must be deterministic when enabled and absent when disabled; custom fee address behavior should not enter consensus accidentally. | Covered for snapshot construction and payment removal; payout-output validation is covered indirectly through share verification paths. | <code>SupportFeeSnapshotsUseCanonicalSlo</code><br><code>SupportFeePaymentRemovesOnlyActual</code> and share-validation tests in <code>boot.tests/ShareAttributionTests.c</code><br>Next: add explicit malformed custom-support-output rejection test vector for external implementers.                                                                                        |
| State bundle sync cannot smuggle stale proofs          | A peer bundle must include enough snapshot context to validate unpaid proofs, and invalid or unrecoverable proofs must be dropped.                                    | Covered for schema gating, missing contexts, invalid recovered contexts, and long-lived reserve import.                                 | <code>CandidateStateWithMissingStateBund</code><br><code>CandidateBundleIncludesSnapshotCon</code><br><code>LoadDropsUnrecoverableWorkSetProof</code><br><code>LoadDropsWorkSetProofsThatDoNotVal</code><br><code>PeerCanImportCandidateStateWithLon</code> in <code>boot.tests/ShareAttributionTests.c</code><br>Next: promote these into portable spec/test-vector fixtures. |

## 16 Remaining Engineering Work

Before broader packaged deployment, the remaining test work is less about missing single-node invariants and more about field confidence:

1. run a clean multi-node V2.1 soak with Main, Dallas, and Evomining on the same release;
2. collect live relay telemetry: peer observation latency, share relay path, and snapshot-boundary disagreement count;
3. add two-process peer-sync regressions for merge-forward and stale-boundary quarantine;
4. convert the most important V2.1 fixtures into portable protocol test vectors for independent implementations;
5. add an explicit malformed custom-support-output rejection vector for external implementers.

## 17 Economic Intuition

GridPool's incentive structure is different from both centralized pools and P2Pool-style sharechains.

In a centralized pool, miners submit shares to an operator-controlled ledger. The operator decides accounting policy, payout timing, transaction-fee treatment, and solvency risk. GridPool replaces that trusted ledger with explicit proof-of-work claims that can be independently validated against payout snapshots.

In P2Pool-style systems, miners compete to extend a fast sharechain. That can make relay latency a continuous orphan-pressure problem: miners who propagate shares faster can repeatedly improve their chance of being on the live share tip. V2.1 does not use a continuous sharechain. Bitcoin blocks create payout snapshot boundaries. A late previous-parent proof may miss a specific snapshot, but it cannot rewrite that snapshot after the boundary; valid current-parent work can merge forward.

The main incentives are:

- slot 0 and transaction fees reward publishing a valid GridPool block rather than withholding it;
- shared payout slots reward previously contributed high-difficulty proofs;
- paid proof IDs are removed after payment, preventing repeated payout of the same claim;
- unpaid reserve proofs carry forward, reducing all-or-nothing payout cliffs;
- there is no trusted pool operator ledger and no continuous sharechain race.

This is why the block-withholding model is important: the finder gives up the most immediate reward channel by withholding. It is also why the pool-hopping model is nuanced: already-earned proof claims are real claims, but hopping cannot create new claims or steal existing ones.

## 18 Reproducibility Appendix

Repository:

```
/home/keegreil/Documents/GitHub/gridpool-simulations
```

The simulation repo was in a local working state when this draft was built. The current base commit reported by `git rev-parse --short HEAD` was:

```
934521e
```

The working tree also included uncommitted paper, model, and plotting changes. For exact reproduction, use the checked-out local tree that contains this paper and the generated report directories listed below.

### 18.1 Payout Variance

```
python3 run_sweep.py \  
--sweep sweeps/payout_variance_fee_sweep.json \  
--out-dir reports/generated/sweeps/payout_variance_fee_long \  
--jobs 4 \  
--heartbeat-seconds 60  
  
python3 plot_sweep_results.py \  
--csv reports/generated/sweeps/payout_variance_fee_long/sweep_results.csv \  
--out-dir reports/generated/sweeps/payout_variance_fee_long/charts
```

Used in this paper:

- reports/generated/sweeps/payout\_variance\_fee\_long/report.md
- reports/generated/sweeps/payout\_variance\_fee\_long/sweep\_results.csv

### 18.2 Pool Hopping

```
python3 run_sweep.py \  
--sweep sweeps/pool_hopping_external_mode_299_sweep.json \  
--out-dir reports/generated/sweeps/pool_hopping_external_mode_299_long
```



```
--jobs 4 \  
--heartbeat-seconds 60
```

```
python3 analyze_pool_hopping_sweep.py \  
--csv reports/generated/sweeps/pool_hopping_external_mode_299_long/sweep_results.csv \  
--out reports/generated/sweeps/pool_hopping_external_mode_299_long/analysis.md
```

```
python3 plot_sweep_results.py \  
--csv reports/generated/sweeps/pool_hopping_external_mode_299_long/sweep_results.csv \  
--out-dir reports/generated/sweeps/pool_hopping_external_mode_299_long/charts
```

Used in this paper:

- reports/generated/sweeps/pool\_hopping\_external\_mode\_299\_long/analysis.md
- reports/generated/sweeps/pool\_hopping\_external\_mode\_299\_long/sweep\_results.csv

### 18.3 Block Withholding

```
python3 run_sweep.py \  
--sweep sweeps/block_withholding_fee_sensitivity_sweep.json \  
--out-dir reports/generated/sweeps/block_withholding_fee_sensitivity_long \  
--jobs 4 \  
--heartbeat-seconds 60
```

```
python3 plot_sweep_results.py \  
--csv reports/generated/sweeps/block_withholding_fee_sensitivity_long/sweep_results.csv \  
--out-dir reports/generated/sweeps/block_withholding_fee_sensitivity_long/charts
```

Used in this paper:

- reports/generated/sweeps/block\_withholding\_fee\_sensitivity\_long/report.md
- reports/generated/sweeps/block\_withholding\_fee\_sensitivity\_long/sweep\_results.csv

### 18.4 Relay Latency

```
python3 run_sweep.py \  
--sweep sweeps/latency_peer_degree_sweep.json \  
--out-dir reports/generated/sweeps/latency_peer_degree_long \  
--jobs 4 \  
--heartbeat-seconds 60
```

```
python3 plot_sweep_results.py \  
--csv reports/generated/sweeps/latency_peer_degree_long/sweep_results.csv \  
--out-dir reports/generated/sweeps/latency_peer_degree_long/charts
```

Used in this paper:

- reports/generated/sweeps/latency\_peer\_degree\_long/report.md
- reports/generated/sweeps/latency\_peer\_degree\_long/sweep\_results.csv

### 18.5 Consensus Scoring Audit

```
python3 run_consensus_selection_audit.py \  
--out-dir reports/generated/consensus_selection_wide_long \  
--trials 10000 \  
--profiles honest,minority_floor_flood,minority_reserve_fill \  
--jobs 4 \  
--heartbeat-seconds 60
```

Used in this paper:

- reports/generated/consensus\_selection\_wide\_long/report.md
- reports/generated/consensus\_selection\_wide\_long/summary\_by\_rule.csv
- reports/generated/consensus\_selection\_eligibility\_long/report.md

## 18.6 V2.1 Boundary-Inclusion Model

```
python3 run_v21_latency_recovery.py \
  --out-dir reports/generated/v21_latency_recovery_july17_long \
  --blocks 5000 \
  --replications 12 \
  --node-count 24 \
  --peer-degree 8
```

Used in this paper:

- reports/generated/v21\_latency\_recovery\_july17\_long/report.md
- reports/generated/v21\_latency\_recovery\_july17\_long/summary.json
- reports/generated/v21\_latency\_recovery\_july17\_long/v21\_latency\_boundary\_metrics.csv

## 19 Public Network Summary

This section is a field sanity check, not statistical proof. Its purpose is to show that GridPool is running beyond a single laptop and to define the public-node topology that future live telemetry should measure. The modeling sections above remain the primary evidence for variance, incentives, and V2.1 boundary behavior.

| Node    | Region                | Role                | Operator         | Current Paper Status                                                 | Next Evidence To Collect                                                                                                                                              |
|---------|-----------------------|---------------------|------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Main    | Washington, D.C. area | Primary public node | Project operator | Existing public node and reference point for state/telemetry checks. | Confirm node version, <code>currentStateId</code> / <code>candidateStateId</code> , peer list, relay observations, and UDP/WebSocket/HTTP counts over a clean window. |
| Dallas  | Dallas, Texas         | Primary public node | Project operator | Existing public node and second primary comparison point.            | Confirm state agreement with Main, peer relay latency, duplicate/accepted observations, and snapshot-boundary disagreement count.                                     |
| Detroit | Detroit, Michigan     | Primary public node | Project operator | Planned; expected online after this draft.                           | Add endpoint, node version, sync status, peer health, and relay telemetry after at least one clean run window.                                                        |

| Node      | Region | Role                  | Operator                     | Current Paper Status                                                                          | Next Evidence To Collect                                                                                          |
|-----------|--------|-----------------------|------------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Evomining | Texas  | Private operator node | Independent/private operator | Existing non-primary public-network participant; useful as an external operator sanity check. | Confirm state convergence, reachable endpoint behavior, peer freshness, and whether UDP observations are visible. |

The final paper should update this table after the Detroit node is online and after a clean telemetry window. Useful live fields are:

- node version and uptime;
- current/candidate state agreement;
- observed peers and duplicate/stale peer entries;
- relay transport counts for UDP, WebSocket, and HTTP JSON;
- accepted versus duplicate/rejected share observations;
- payload sizes by transport;
- snapshot-boundary disagreement count.

Until those measurements are clean, this section should be read as operational context rather than a production latency benchmark.

## 20 Resource Roadmap

Additional resources should be tied directly to risk reduction and delivery acceleration.

| Resource                         | What It Unlocks                                                                                                                           | Risk Reduced                                                                                              |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Larger simulation budget         | More Monte Carlo trials, wider parameter sweeps, and cleaner confidence intervals for variance, withholding, hopping, and latency models. | Reduces risk that published claims depend on narrow parameter choices or noisy runs.                      |
| Multi-region public-node testnet | Always-on nodes across several networks, with controlled versions and reproducible telemetry windows.                                     | Reduces uncertainty around real relay latency, NAT/firewall behavior, and snapshot-boundary disagreement. |
| Adversarial regression suite     | Automated tests for stale bundles, malformed payout outputs, duplicate proof IDs, delayed proofs, and peer sync edge cases.               | Reduces risk of reintroducing stale-branch rewrite or paid-lineage bugs.                                  |
| Portable protocol test vectors   | JSON fixtures for payout snapshots, Work Set imports, paid-lineage removal, support-fee on/off behavior, and rejected stale proofs.       | Helps independent implementers verify compatibility without trusting the reference implementation.        |
| Compact/UDP relay hardening      | Better transport fallback, observability, packet-size control, and public-node diagnostics.                                               | Reduces propagation latency and improves detection of relay failures.                                     |
| Umbrel/Start9 packaging          | Installable node packages with versioned config, service management, and health checks.                                                   | Reduces deployment friction and makes wider public testing possible.                                      |

| Resource                   | What It Unlocks                                                                                             | Risk Reduced                                                               |
|----------------------------|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Public telemetry dashboard | Snapshot state, peer health, relay observations, rejected stale proofs, and boundary-disagreement counters. | Makes live behavior auditable by developers and funders.                   |
| Third-party review budget  | External review of consensus rules, payout construction, and adversarial test coverage.                     | Reduces blind spots from internal modeling and implementation assumptions. |

## 21 Source Pointers

Primary docs:

- `reports/july17/gridpool-v2.1-consensus-note.md`
- `docs/HANDOFF-2026-07-10.md`
- `docs/consensus-selection-audit-results-2026-06.md`
- `docs/live-relay-telemetry-plan.md`

Primary generated reports:

- `reports/generated/sweeps/payout_variance_fee_long/report.md`
- `reports/generated/sweeps/pool_hopping_external_mode_299_long/analysis.md`
- `reports/generated/sweeps/block_withholding_fee_sensitivity_long/report.md`
- `reports/generated/sweeps/latency_peer_degree_long/report.md`
- `reports/generated/v21_latency_recovery_july17_long/report.md`
- `reports/generated/consensus_selection_wide_long/report.md`
- `reports/generated/consensus_selection_eligibility_long/report.md`
- `reports/generated/delayed_snapshot_attack_survival_poolshare_0.001/report.md`